

OpenShift Full Detailed Learner Guide

Concepts, Architecture, CLI/GUI Flow, Labs, Diagrams, Troubleshooting and Best Practices

Prepared for classroom and hands-on sandbox delivery

Prepared by: Vivek Arora

Senior Technical Trainer & Author/Consultant

Date: 08 May 2026

Guide coverage:

- OpenShift native objects, builds, ImageStreams and deployments
- Configuration, networking, storage, security, scaling, monitoring and logging
- RBAC, service accounts, CI/CD, GitOps, cloud-native operations and cost awareness
- Step-by-step labs for OpenShift Developer Sandbox
- Common errors from the hands-on session and how to fix them

How to Use This Learner Guide

This guide consolidates the full OpenShift discussion and hands-on flow covered in this chat. It is written as a learner guide, so each topic includes concept explanation, trainer notes, diagrams, CLI commands, GUI navigation where applicable, lab steps, common errors, and cleanup guidance.

Sandbox note: Red Hat OpenShift Developer Sandbox often restricts cluster-admin operations, project creation, SCC assignment, NodePort reachability, cluster upgrades, and full logging/monitoring installation. Labs are written to work with normal project access wherever possible.

Recommended learning sequence

Phase	Topics
Foundation	OpenShift context, projects, pods, deployments, services, routes
Build and release	BuildConfig, S2I, Docker builds, ImageStreams, tagging, triggers, rollback
Day-2 operations	Configuration, networking, storage, security, scaling, monitoring, RBAC
Delivery and cloud	Pipelines/Tekton, Jenkins, GitOps/Argo CD, cloud OpenShift, upgrades, cost optimization

Core troubleshooting mindset

When something fails in OpenShift, follow this order:

1. `oc get <resource>`
2. `oc describe <resource>`
3. Check Events section
4. Check logs if a pod/build/task exists
5. Check quota and permissions
6. Clean old lab resources if quota is full

Table of Contents

1. OpenShift CLI, contexts and common terminal mistakes
2. Native OpenShift objects - BuildConfig, ImageStream, DeploymentConfig
3. Build and image management - internal registry, S2I, Docker, custom build, triggers
4. Application lifecycle - rollouts, rollback, recreate, rolling, blue-green
5. Configuration management - ConfigMaps, Secrets, env variables, mounted files, dynamic updates
6. Networking and exposure - Services, Routes, Ingress, TLS termination
7. Storage and persistence - PV, PVC, StorageClass, dynamic provisioning, stateful vs stateless
8. Security model - SCC, non-root containers, secure image practices, trusted registries
9. Scaling and reliability - HPA, requests/limits, high availability, disruption basics
10. Monitoring and logging - logs, metrics, alerts, PromQL, UI flow
11. Access control - RBAC and service accounts
12. CI/CD - OpenShift Pipelines/Tekton, Build-Test-Deploy, Git integration
13. DevOps and GitOps - Jenkins, Argo CD, image promotion
14. Cloud-native OpenShift - AWS/Azure/GCP, upgrades, operators
15. Optimization and cost awareness
16. Troubleshooting appendix and cleanup commands

1. OpenShift CLI, Contexts and Terminal Basics

Learning objective: Understand what oc does, how context maps CLI to cluster/project, and how to avoid common shell mistakes.

Concept flow

OpenShift Web Console -> OpenShift API Server
oc CLI -> OpenShift API Server

Context = Cluster + User + Project/Namespace

What is oc?

oc is the OpenShift command-line interface. It can manage Kubernetes resources such as Pods, Deployments and Services, and OpenShift-specific resources such as Routes, BuildConfigs, ImageStreams and DeploymentConfigs.

Command	Purpose
oc login	Login to an OpenShift cluster using token and API server.
oc whoami	Show current logged-in user.
oc whoami --show-server	Show current OpenShift API server.
oc project	Show or switch current project/namespace.
oc get all	List common project resources.
oc apply -f file.yaml	Create or update resources from YAML.
oc logs <pod>	View pod logs.
oc exec -it <pod> -- /bin/sh	Enter a running container shell.

Meaning of context

When the CLI says: Using project "vivek-c-dev" from context named "vivek-c-context" on server "https://172.30.0.1:443", it means the active kubeconfig context points to a specific cluster, user and project.

```
oc config current-context
oc config get-contexts
oc project -q
oc whoami --show-server
```

Inside the OpenShift Web Terminal, the API server may show as https://172.30.0.1:443. This is internal to the cluster and is fine inside the web terminal. From a local laptop, use the public API server copied from the console login command.

Common Bash variable mistake

Bash variable assignment must not have spaces around equals sign.

```
# Wrong
PROJECT = $(oc project -q) echo $PROJECT
```

```
# Correct
PROJECT=$(oc project -q)
echo $PROJECT
```

```
# One line
PROJECT=$(oc project -q); echo $PROJECT
```

Inside container vs OpenShift terminal

When the prompt is bash-5.1 ~ \$, you are in the OpenShift terminal and oc normally works. When the prompt is sh-5.1\$, you are inside a container. Most application containers do not include oc.

```
# From OpenShift terminal
```

```
oc exec -it pvc-writer-pod -- /bin/sh

# Now inside container; use Linux commands, not oc
ls -l /data
cat /data/storage-test.txt
exit
```

2. OpenShift Native Objects

Learning objective: Explain BuildConfig, ImageStream, DeploymentConfig and how they differ from Kubernetes-native Deployment.

Concept flow

Source Code -> BuildConfig -> Build Pod -> Image -> Internal Registry
Internal Registry -> ImageStreamTag -> DeploymentConfig/Deployment -> Pods

BuildConfig

BuildConfig is an OpenShift object that defines how source code or binary input is converted into a container image. It is like a build recipe or factory plan.

BuildConfig Part	Meaning
source	Git repo, binary upload, Dockerfile context or image source.
strategy	S2I, Docker or Custom build strategy.
output	Usually an ImageStreamTag such as app:latest.
triggers	Manual, config change, image change or webhook.
build pod	Temporary pod that runs the build.

```
oc get bc
oc describe bc hello-s2i
oc start-build hello-s2i --follow
oc logs -f bc/hello-s2i
```

ImageStream - very important

An ImageStream is not the actual image data. It is an OpenShift tracking object that stores image tags and references to image digests. The actual image may be in the OpenShift internal registry or in an external registry.

Memory point: ImageStream = tracker/bookmark. Internal registry = image storage.

Scenario	Goes to internal registry?	Reason
oc create deployment app --image=nginx	No	Pod pulls external image; it is not stored in internal registry.
oc new-app nodejs~https://github.com/...	Yes	S2I build creates a new image and pushes output to internal registry.
oc new-build --strategy=docker --binary	Yes if output is ImageStreamTag	Docker strategy build output is pushed to the configured output.
oc import-image nginx --from=docker.io/library/nginx	Mostly tracking	ImageStream tracks external image metadata.
oc tag app:latest app:prod	No new copy usually	Tag pointer/promotion behavior.
podman push image-registry.../project/app	Yes	You explicitly push to internal registry.

```
oc get is
oc describe is hello-s2i
oc get is hello-s2i -o jsonpath='{.status.dockerImageRepository}'
echo
oc get istag
```

DeploymentConfig vs Deployment

Deployment is Kubernetes-native and manages ReplicaSets. DeploymentConfig is OpenShift-native and manages ReplicationControllers. DeploymentConfig supports OpenShift-native deployment triggers, especially ImageStream triggers.

Feature	Deployment	DeploymentConfig
Origin	Kubernetes native	OpenShift native
Managed object	ReplicaSet	ReplicationController
ImageStream trigger	Not native by default	Native support
Rollout command	oc rollout ... deployment/name	oc rollout ... dc/name
Recommended usage	Modern Kubernetes apps	OpenShift-native trigger/hook scenarios

```
oc get deployment
oc get dc
oc describe dc hello-s2i
oc rollout history dc/hello-s2i
```

3. Build and Image Management

Learning objective: Understand internal registry behavior, image tagging, S2I, Docker build, Custom build and build triggers.

Concept flow

```
Git Source / Binary Input / Dockerfile
-> BuildConfig
-> Build Pod
-> New Image
-> Internal Registry
-> ImageStreamTag
-> Deployment Trigger
```

Internal registry working

The OpenShift internal registry stores images built inside the cluster. Pods can pull those images using service account permissions. Public images used directly are not automatically copied into the internal registry.

```
# Direct external image example - does not save to internal registry
oc create deployment public-demo --image=quay.io/openshift/origin-hello-openshift:latest
oc get deployment public-demo -o jsonpath='{.spec.template.spec.containers[0].image}'
echo
```

```
# S2I build example - output goes to internal registry
oc new-app nodejs~https://github.com/sclorg/nodejs-ex --name=hello-s2i
  --as-deployment-config=true
oc get is hello-s2i -o jsonpath='{.status.dockerImageRepository}'
echo
```

Image tagging and promotion

Image promotion means moving a tested image through tags such as latest, dev, test and prod without rebuilding every time.

```
oc tag pipeline-demo:latest pipeline-demo:dev
oc tag pipeline-demo:dev pipeline-demo:test
oc tag pipeline-demo:test pipeline-demo:prod
oc describe is pipeline-demo
oc get istag
```

S2I build strategy

Source-to-Image, or S2I, takes application source code and a builder image and produces a runnable application image. It is developer-friendly because the developer may not need to write a Dockerfile.

```
oc new-app nodejs~https://github.com/sclorg/nodejs-ex --name=hello-s2i
oc get bc
oc get builds
oc logs -f bc/hello-s2i
oc start-build hello-s2i --follow
```

Docker build strategy

Docker strategy uses a Dockerfile-based build. In OpenShift 4, the build process is commonly implemented using Buildah in the build pod rather than a Docker daemon.

```
cat > Dockerfile <<'EOF'
FROM registry.access.redhat.com/ubi9/ubi-minimal
RUN microdnf install -y httpd && microdnf clean all
RUN echo "Hello from Docker Build Strategy on OpenShift" > /var/www/html/index.html
EXPOSE 8080
CMD ["httpd", "-D", "FOREGROUND"]
EOF
```

```
oc new-build --strategy=docker --binary --name=docker-demo
oc start-build docker-demo --from-dir=. --follow
```

In Sandbox, Docker builds may take time. If --follow times out but upload finished, check `oc get builds`, `oc describe build <name>`, and `oc logs -f build/<name>`.

Dockerfile security-friendly example

```
FROM registry.access.redhat.com/ubi9/ubi-minimal

WORKDIR /app
COPY app.sh /app/app.sh

RUN chmod +x /app/app.sh && chmod -R g=u /app
USER 1001

CMD ["/app/app.sh"]
```

Custom build strategy

Custom builds use a custom builder image that controls the complete build process. They are powerful but generally not available to normal Sandbox users because they may require elevated permissions.

```
apiVersion: build.openshift.io/v1
kind: BuildConfig
metadata:
  name: custom-build-demo
spec:
  source:
    git:
      uri: https://github.com/example/custom-source
  strategy:
    customStrategy:
      from:
        kind: DockerImage
        name: custom-builder-image:latest
  output:
    to:
      kind: ImageStreamTag
      name: custom-build-demo:latest
```

Build triggers

Trigger	Meaning
Manual	User starts build using <code>oc start-build</code> .
ConfigChange	Build starts when BuildConfig changes.
ImageChange	Build starts when base/builder image changes.

Trigger	Meaning
GitHub/GitLab webhook	Build starts after Git push event.
Generic webhook	External system can trigger the build.

```
oc describe bc hello-s2i
oc set triggers bc/hello-s2i --list
oc start-build hello-s2i --follow
oc set env bc/hello-s2i TRAINING_DEMO=true
```

GitHub webhook UI

In GitHub, go to Repository -> Settings -> Webhooks -> Add webhook. Payload URL must be the full OpenShift BuildConfig webhook URL and must start with https://. If GitHub says missing scheme, the URL is missing http:// or https://.

```
# Get webhook URL
oc describe bc hello-s2i
```

```
# Common mistake: using internal API URL from Web Terminal
https://172.30.0.1:443/... # GitHub cannot reach this from internet
```

```
# Better: use public API URL copied from OpenShift console login command
```

4. Application Lifecycle Management

Learning objective: Manage versions, rollbacks, deployment strategies and blue-green releases.

Concept flow

```
Build 1 -> Image v1 -> Deployment Revision 1
Build 2 -> Image v2 -> Deployment Revision 2
Issue found -> Rollback to older revision or older image tag
```

Versioning and rollback

```
# DeploymentConfig
oc rollout history dc/hello-s2i
oc rollout status dc/hello-s2i
oc rollback hello-s2i
oc rollout latest dc/hello-s2i

# Kubernetes Deployment
oc rollout history deployment/<deployment-name>
oc rollout undo deployment/<deployment-name>
oc rollout status deployment/<deployment-name>
```

Rolling vs Recreate strategy

Strategy	Behavior	Best for
Rolling	Starts new pods gradually and removes old pods gradually.	Zero-downtime compatible applications.
Recreate	Stops old pods first, then starts new pods.	Legacy apps or incompatible versions.

```
# Check current strategy
oc get dc hello-s2i -o yaml | grep -A5 strategy
```

```
# Change DeploymentConfig strategy to Recreate
oc patch dc hello-s2i -p '{"spec":{"strategy":{"type":"Recreate"}}}'
```

```
# Manually trigger a new rollout
oc rollout latest dc/hello-s2i
oc rollout status dc/hello-s2i
```

```
# Restore Rolling strategy
oc patch dc hello-s2i -p '{"spec":{"strategy":{"type":"Rolling"}}}'
```


Blue-green deployment

Blue-green means two versions run side by side. Blue is current production. Green is the new version. The route is switched from blue service to green service after validation.

User -> Route -> Blue Service -> Blue Pods

After switch:

User -> Route -> Green Service -> Green Pods

Rollback:

User -> Route -> Blue Service -> Blue Pods

```
oc create deployment vk-blue-app --image=quay.io/openshift/origin-hello-openshift:latest
oc label deployment vk-blue-app app=vk-bg version=blue
oc expose deployment vk-blue-app --port=8080 --name=vk-blue-svc
oc expose svc vk-blue-svc --name=vk-bg-route
```

```
oc create deployment vk-green-app --image=quay.io/openshift/origin-hello-openshift:latest
oc label deployment vk-green-app app=vk-bg version=green
oc expose deployment vk-green-app --port=8080 --name=vk-green-svc
```

Switch route to green. Kind is case-sensitive: Service, not service.

```
oc patch route vk-bg-route -p '{"spec":{"to":{"kind":"Service","name":"vk-green-svc"}}}'
```

Rollback to blue

```
oc patch route vk-bg-route -p '{"spec":{"to":{"kind":"Service","name":"vk-blue-svc"}}}'
```

If you get Invalid value: "service": must reference a Service, fix the case: "Service".

5. Configuration Management

Learning objective: Use ConfigMaps and Secrets as environment variables and mounted files, and understand dynamic update behavior.

Concept flow

```
ConfigMap/Secret -> Pod
    -> Environment variables
    -> Mounted files
    -> Application reads configuration
```

Why external configuration?

The same container image should run in dev, test and prod. Environment-specific values should be injected at runtime using ConfigMaps and Secrets.

Use ConfigMap for	Use Secret for
APP_ENV, LOG_LEVEL, API URL, feature flag, application.properties	DB_PASSWORD, API_TOKEN, private key, TLS certificate

Create ConfigMap from literals

```
oc create configmap app-config --from-literal=APP_ENV=dev --from-literal=LOG_LEVEL=debug
--from-literal=APP_VERSION=1.0
```

```
oc get cm
oc describe cm app-config
oc get cm app-config -o yaml
```

ConfigMap as environment variables

```
cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: config-env-pod
  labels:
```

```

    app: config-env-demo
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      while true;
      do
        echo "APP_ENV=$APP_ENV";
        echo "LOG_LEVEL=$LOG_LEVEL";
        echo "APP_VERSION=$APP_VERSION";
        sleep 30;
      done
    envFrom:
    - configMapRef:
        name: app-config
    restartPolicy: Never
EOF

```

The key line is envFrom -> configMapRef -> name: app-config. All keys in the ConfigMap become environment variables inside the container.

```

oc get pod config-env-pod
oc logs config-env-pod
oc exec -it config-env-pod -- env | grep -E 'APP_ENV|LOG_LEVEL|APP_VERSION'

```

ConfigMap as mounted file

Some applications expect files such as application.properties, application.yml, nginx.conf, prometheus.yml or log4j.properties. In that case, store the file in a ConfigMap and mount it as a volume.

```

cat > application.properties <<'EOF'
server.port=8080
app.name=openshift-config-demo
app.environment=dev
logging.level=DEBUG
feature.payment.enabled=true
EOF

oc create configmap app-file-config --from-file=application.properties
oc describe cm app-file-config

cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: config-volume-pod
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      while true;
      do
        echo "Reading mounted config file:";
        cat /etc/app-config/application.properties;
        echo "-----";
        sleep 30;
      done
    volumeMounts:
    - name: app-config-volume
      mountPath: /etc/app-config
  volumes:
  - name: app-config-volume
    configMap:
      name: app-file-config
    restartPolicy: Never
EOF

```

The ConfigMap key application.properties becomes the file /etc/app-config/application.properties inside the container.

```
oc logs config-volume-pod
```

```
oc exec -it config-volume-pod -- cat /etc/app-config/application.properties
```

Secrets secure handling

Secrets should store sensitive data. They are base64-encoded by default, but base64 is not encryption. Avoid printing secret values in logs or committing Secret YAML with real values to Git.

```
oc create secret generic db-secret --from-literal=DB_USER=admin
--from-literal=DB_PASSWORD='redhat123'
```

```
oc get secret
oc describe secret db-secret
oc get secret db-secret -o yaml
```

Secret as environment variable

```
cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      while true;
      do
        echo "DB_USER=$DB_USER";
        echo "DB_PASSWORD is available but should not be printed";
        sleep 30;
      done
    env:
    - name: DB_USER
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: DB_USER
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: DB_PASSWORD
    restartPolicy: Never
EOF
```

Secret as mounted file

```
cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: secret-volume-pod
  labels:
    app: secret-volume-demo
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      while true;
      do
        echo "Secret files available:";
        ls -l /etc/secrets;
        echo "DB_USER from file:";
        cat /etc/secrets/DB_USER;
        sleep 30;
      done
    volumeMounts:
    - name: db-secret-volume
      mountPath: /etc/secrets
      readOnly: true
  volumes:
  - name: db-secret-volume
```

```

secret:
  secretName: db-secret
  restartPolicy: Never
EOF

```

Secret key	File path inside container	File content
DB_USER	/etc/secrets/DB_USER	admin
DB_PASSWORD	/etc/secrets/DB_PASSWORD	redhat123

Environment variables vs mounted configs

Area	Environment variables	Mounted files
Best for	Simple key-value values	Full configuration files, certs, keys
Dynamic update	No automatic update; recreate/restart pod	File can update after a delay
Access method	System.getenv or shell env	Read from filesystem path
Large config	Not ideal	Better
Security	May appear in env/process dump	Good for certificate/key files

Dynamic config updates

```

# Mounted ConfigMap file can update after some time
oc create configmap app-file-config
  --from-literal=application.properties='server.port=8080'
app.name=openshift-config-demo
app.environment=dev
logging.level=INFO
feature.payment.enabled=false' --dry-run=client -o yaml | oc apply -f -

oc exec -it config-volume-pod -- cat /etc/app-config/application.properties

# Env variable config does not update in running container
oc patch configmap app-config -p
  '{"data":{"APP_ENV":"dev","LOG_LEVEL":"info","APP_VERSION":"2.0"}}'

oc delete pod config-env-pod
# Recreate pod to see new env values

```

6. Networking and Exposure

Learning objective: Understand how Pods are exposed internally and externally using Services, Routes, Ingress and TLS termination.

Concept flow

User -> Route/Ingress -> Service -> Pod

Service selector must match Pod label for traffic to flow.

Service basics

Pods get temporary IPs. A Service gives stable access to pods selected by labels. Service selector and pod labels must match.

```

cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: network-demo-pod
  labels:
    app: network-demo
spec:
  containers:

```

```

- name: app
  image: quay.io/openshift/origin-hello-openshift:latest
  ports:
    - containerPort: 8080
  restartPolicy: Never
EOF

cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Service
metadata:
  name: network-demo-svc
spec:
  type: ClusterIP
  selector:
    app: network-demo
  ports:
    - name: http
      port: 8080
      targetPort: 8080
EOF

oc get svc network-demo-svc
oc get endpoints network-demo-svc
curl http://network-demo-svc:8080
curl http://network-demo-svc.$(oc project -q).svc.cluster.local:8080

```

ClusterIP and NodePort

Service Type	Meaning	OpenShift usage
ClusterIP	Internal cluster service, default type.	Common for app-to-app communication.
NodePort	Exposes service on a port on every node.	Usually not preferred; may not be reachable in Sandbox.

Routes - OpenShift-specific external exposure

A Route exposes a Service externally using a hostname through the OpenShift router.

```

oc expose svc/network-demo-svc --name=network-demo-route
oc get route
oc get route network-demo-route -o jsonpath='{.spec.host}'
echo
curl http://$(oc get route network-demo-route -o jsonpath='{.spec.host}')

```

Ingress vs Route

Feature	Ingress	Route
Origin	Kubernetes native	OpenShift specific
Controller	Ingress controller	OpenShift router
Usage in OpenShift	Supported	Very common/native
TLS	Supported depending on controller	Native route TLS termination options

TLS termination types

Type	Client to Router	Router to Pod	Router decrypts?	Backend needs TLS?
Edge	HTTPS	HTTP	Yes	No
Re-encrypt	HTTPS	HTTPS	Yes	Yes
Passthrough	HTTPS	HTTPS stream	No	Yes

```

# Edge TLS route
oc delete route network-demo-route --ignore-not-found
oc create route edge network-demo-route --service=network-demo-svc --port=8080

```

```
curl -k https://$(oc get route network-demo-route -o jsonpath='{.spec.host}')
```

Conceptual re-encrypt route - backend must serve HTTPS

```
oc create route reencrypt secure-app-route --service=secure-app-svc --port=8443
```

Conceptual passthrough route - backend terminates TLS

```
oc create route passthrough secure-pass-route --service=secure-app-svc --port=8443
```

Networking troubleshooting

```
oc get pods --show-labels
oc describe svc network-demo-svc
oc get endpoints network-demo-svc
oc describe route network-demo-route
```

Route gives 503? Usually service has no endpoint or pod is not ready.
Endpoint empty? Selector does not match pod labels.

7. Storage and Persistence

Learning objective: Understand PV, PVC, StorageClass, dynamic provisioning and stateful vs stateless applications.

Concept flow

Pod -> PVC -> PV -> Actual storage

PVC = request for storage
PV = actual storage resource
StorageClass = dynamic provisioning template

Stateless vs stateful

Area	Stateless app	Stateful app
Examples	Frontend, REST API, gateway	PostgreSQL, MySQL, Kafka, Elasticsearch
Needs persistent storage?	Usually no	Yes
Pod identity	Any pod can replace another	Stable identity often required
Recommended controller	Deployment	StatefulSet

PersistentVolume and PersistentVolumeClaim

A PersistentVolume is actual storage available in the cluster. A PersistentVolumeClaim is a request for storage from a project. In Sandbox, you normally create PVCs; PV creation is cluster-level and often restricted.

```
oc get sc
oc describe sc <storageclass-name>
```

```
cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: demo-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
EOF
```

```
oc get pvc
oc describe pvc demo-pvc
```

Pod using PVC

```
cat <<'EOF' | oc apply -f -
```

```

apiVersion: v1
kind: Pod
metadata:
  name: pvc-writer-pod
spec:
  containers:
    - name: app
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      command: ["/bin/sh", "-c"]
      args:
        - |
          echo "Writing data to persistent volume...";
          date >> /data/storage-test.txt;
          echo "Pod name: $HOSTNAME" >> /data/storage-test.txt;
          while true;
          do
            echo "Current file content:";
            cat /data/storage-test.txt;
            sleep 30;
          done
      volumeMounts:
        - name: demo-storage
          mountPath: /data
  volumes:
    - name: demo-storage
      persistentVolumeClaim:
        claimName: demo-pvc
      restartPolicy: Never
EOF

oc logs pvc-writer-pod
oc exec -it pvc-writer-pod -- /bin/sh
ls -l /data
cat /data/storage-test.txt
exit

```

Prove persistence

```

oc delete pod pvc-writer-pod
oc get pvc demo-pvc

```

```

# Create a new pod using the same PVC and read /data/storage-test.txt.
# Data survives because it lives on the volume, not inside the old container.

```

Access modes and dynamic provisioning

Access Mode	Meaning	Typical usage
ReadWriteOnce (RWO)	Read-write by one node.	Most block storage/database labs.
ReadOnlyMany (ROX)	Read-only by many nodes.	Shared static content.
ReadWriteMany (RWX)	Read-write by many nodes.	Shared file storage.
ReadWriteOncePod (RWOP)	Read-write by one pod only.	Strict single-writer workloads.

Dynamic provisioning means the PV is automatically created when the PVC is created, using the selected StorageClass.

8. OpenShift Security Model

Learning objective: Understand SCC, non-root containers, secure image practices and trusted registries.

Concept flow

```

Pod YAML -> ServiceAccount -> SCC admission check
Allowed -> Pod runs with restricted permissions
Rejected -> Error: unable to validate against any security context constraint

```

Security Context Constraints

SCC is an OpenShift-specific security rulebook that decides whether a pod is allowed to run with requested security settings such as root user, privileged mode, host networking, hostPath volumes and Linux capabilities.

SCC	Meaning
restricted/restricted-v2	Default safer SCC for normal workloads.
anyuid	Allows any UID including root.
privileged	Allows highly privileged containers.
nonroot	Requires non-root user.
hostnetwork	Allows host networking.

```
oc auth can-i use scc/anyuid
oc auth can-i use scc/privileged
oc auth can-i create pods
```

Non-root compatible pod

```
cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: security-nonroot-pod
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      while true;
      do
        echo "Container is running";
        id;
        sleep 30;
      done
  restartPolicy: Never
EOF
```

```
oc logs security-nonroot-pod
oc exec -it security-nonroot-pod -- id
```

Root and privileged pod rejection examples

```
# Run as root - often rejected in restricted projects
securityContext:
  runAsUser: 0

# Privileged - should be rejected in Sandbox
securityContext:
  privileged: true

# Check failure details
oc describe pod <pod-name>
```

Secure image practices

Practice	Why
Use minimal base images	Less attack surface and smaller pulls.
Run as non-root	Reduces compromise impact.
Avoid latest in production	Prevents uncontrolled version changes.
Scan images	Find vulnerabilities before deployment.
Do not bake secrets into images	Avoid credential leakage.
Use trusted registries	Reduce supply-chain risk.

Practice	Why
Pin version or digest	Improve reproducibility.

```
# Secure-ish pod securityContext concept
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
  capabilities:
    drop:
      - ALL
```

9. Scaling and Reliability

Learning objective: Use resource requests/limits, HPA, multiple replicas and PodDisruptionBudget concepts.

Concept flow

Route -> Service -> Multiple Pods
 HPA observes metrics -> updates replica count
 PDB protects minimum availability during voluntary disruption

Resource requests and limits

Item	Meaning	Impact
CPU request	Minimum CPU used for scheduling.	Required for CPU-based HPA percentage.
CPU limit	Maximum CPU allowed.	Excess CPU is throttled.
Memory request	Minimum memory used for scheduling.	Helps node placement.
Memory limit	Maximum memory allowed.	Exceeding can cause OOMKilled.

```
resources:
  requests:
    cpu: "50m"
    memory: "64Mi"
  limits:
    cpu: "100m"
    memory: "128Mi"
```

Resource demo Deployment

```
cat <<'EOF' | oc apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: resource-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: resource-demo
  template:
    metadata:
      labels:
        app: resource-demo
    spec:
      containers:
        - name: app
          image: registry.access.redhat.com/ubi9/ubi-minimal:latest
          command: ["/bin/sh", "-c"]
          args:
            - |
              while true;
              do
                echo "Resource demo running";
                sleep 30;
              done
          resources:
            requests:
```

```

      cpu: "50m"
      memory: "64Mi"
    limits:
      cpu: "100m"
      memory: "128Mi"
EOF

```

EOF

Horizontal Pod Autoscaler

HPA increases or decreases the number of pods based on metrics such as CPU utilization. It needs realistic resource requests.

```

oc autoscale deployment hpa-cpu-demo --cpu-percent=50 --min=1 --max=3
oc get hpa
oc describe hpa hpa-cpu-demo
oc get pods -l app=hp-cpu-demo

```

High availability

High availability for stateless apps starts with multiple replicas behind a Service. If one pod fails, the Deployment creates a replacement and Service sends traffic to remaining ready pods.

```

oc scale deployment ha-demo --replicas=3
oc get pods -l app=ha-demo

```

```

# Delete one pod and observe self-healing
oc delete pod <one-ha-demo-pod-name>
oc get pods -l app=ha-demo

```

Pod disruption basics

A PodDisruptionBudget protects against voluntary disruptions such as node drain or maintenance. It does not prevent sudden crashes.

```

cat <<'EOF' | oc apply -f -
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: ha-demo-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: ha-demo
EOF

```

```

oc get pdb
oc describe pdb ha-demo-pdb

```

10. Monitoring and Logging

Learning objective: Use CLI/UI logs, events, metrics, alerts and PromQL to troubleshoot OpenShift workloads.

Concept flow

```

Pod logs -> oc logs / UI Logs
Pod metrics -> Prometheus/OpenShift Observe
Events -> oc describe / Events tab
Alerts -> Prometheus rules -> Alertmanager -> Console

```

Logging via CLI

Task	Command
View pod logs	oc logs <pod-name>
Follow logs live	oc logs -f <pod-name>
Logs by label	oc logs -l app=<label>

Task	Command
Previous crashed container	oc logs <pod-name> --previous
Specific container	oc logs <pod-name> -c <container-name>
Build logs	oc logs -f bc/<buildconfig-name>

```
cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: log-demo-pod
  labels:
    app: log-demo
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      counter=1;
      while true;
      do
        echo "INFO: Application is running. Count=$counter";
        echo "DEBUG: Current time is $(date)";
        if [ $((counter % 5)) -eq 0 ]; then
          echo "WARN: This is a sample warning message";
        fi
        counter=$((counter+1));
        sleep 10;
      done
  restartPolicy: Never
EOF
```

```
oc logs log-demo-pod
oc logs -f log-demo-pod
```

Logging and events via UI

```
OpenShift Console
-> Administrator Perspective
-> Workloads
-> Pods
-> Select project
-> Click Pod
-> Logs tab
-> Events tab
```

Metrics and alerts via GUI

```
Developer Perspective
-> Observe
-> Metrics
-> Select project
-> Paste PromQL query
-> Run Query
```

```
Administrator Perspective
-> Observe
-> Alerting/Alerts
-> Dashboards
-> Targets
```

PromQL syntax

```
metric_name{label="value"}
metric_name{label="value"}[5m]
function(metric_name{label="value"}[5m])
sum(function(metric_name{label="value"}[5m])) by (label)
```

Matcher	Meaning	Example
=	Equal	{namespace="vivek-c-dev"}
!=	Not equal	{container!="POD"}
=~	Regex match	{pod=~"ha-demo.*"}
!~	Regex not match	{pod!~"workspace.*"}

Useful OpenShift PromQL queries

```
# CPU by pod
sum(rate(container_cpu_usage_seconds_total{namespace="vivek-c-dev", container!="POD"}[5m]))
  by (pod)

# Memory by pod in MiB
sum(container_memory_working_set_bytes{namespace="vivek-c-dev", container!="POD"}) by (pod)
  / 1024 / 1024

# Pod restarts
sum(kube_pod_container_status_restarts_total{namespace="vivek-c-dev"}) by (pod)

# Pods by status
sum(kube_pod_status_phase{namespace="vivek-c-dev"}) by (phase)

# Top 5 CPU pods
topk(5, sum(rate(container_cpu_usage_seconds_total{namespace="vivek-c-dev",
  container!="POD"}[5m])) by (pod))

# Top 5 memory pods
topk(5, sum(container_memory_working_set_bytes{namespace="vivek-c-dev", container!="POD"})
  by (pod))
```

Troubleshooting sequence

```
oc get pods
oc describe pod <pod-name>
oc logs <pod-name>
oc logs <pod-name> --previous
oc get events --sort-by=.lastTimestamp
oc top pods
```

11. Access Control Deep Dive

Learning objective: Understand RBAC roles, bindings, service accounts and least-privilege access.

Concept flow

```
Subject (User/Group/ServiceAccount)
  -> RoleBinding
  -> Role/ClusterRole
  -> Permissions on resources
```

RBAC objects

Object	Scope	Purpose
Role	Project/namespace	Defines permissions inside one project.
ClusterRole	Cluster-wide or reusable	Defines cluster permissions or reusable role.
RoleBinding	Project/namespace	Assigns Role/ClusterRole to subject in one project.
ClusterRoleBinding	Cluster-wide	Assigns ClusterRole across cluster.

RBAC verbs

Verb	Meaning
get	View one object
list	List objects
watch	Watch changes
create	Create object
update	Modify full object
patch	Modify part of object
delete	Delete object
deletecollection	Delete multiple objects

Check access

```
oc whoami
oc project -q
oc auth can-i create pods
oc auth can-i create deployments
oc auth can-i create rolebindings
oc auth can-i use scc/privileged
oc auth can-i list secrets
```

Create custom Role and bind to ServiceAccount

```
cat <<'EOF' | oc apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader-role
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
EOF

oc create serviceaccount app-reader

oc create rolebinding app-reader-pod-reader-binding --role=pod-reader-role
--serviceaccount=$(oc project -q):app-reader

oc describe rolebinding app-reader-pod-reader-binding
```

Pod using ServiceAccount

```
cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: sa-demo-pod
spec:
  serviceAccountName: app-reader
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      echo "ServiceAccount demo pod started";
      id;
      ls -l /var/run/secrets/kubernetes.io/serviceaccount || true;
      while true; do sleep 30; done
    restartPolicy: Never
EOF
```

Do not print the service account token in real environments. If a pod does not need API access, set automountServiceAccountToken: false.

```

apiVersion: v1
kind: Pod
metadata:
  name: no-token-pod
spec:
  automountServiceAccountToken: false
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args: ["sleep 3600"]
    restartPolicy: Never

```

12. CI/CD in OpenShift

Learning objective: Understand OpenShift Pipelines/Tekton, Build-Test-Deploy flow and Git integration.

Concept flow

```

Git Repository -> PipelineRun
PipelineRun -> TaskRun: Build
PipelineRun -> TaskRun: Test
PipelineRun -> TaskRun: Deploy
Deploy -> OpenShift Deployment/Route

```

Tekton objects

Object	Meaning
Task	Reusable group of one or more steps.
Step	One container command inside a Task.
Pipeline	Ordered set of Tasks.
PipelineRun	Actual execution of a Pipeline.
TaskRun	Actual execution of a Task.
Workspace	Shared storage between Tasks.
Trigger/EventListener	Webhook-based pipeline trigger.

Simple message Task

```

cat <<'EOF' | oc apply -f -
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: simple-message-task
spec:
  params:
  - name: message
    type: string
  steps:
  - name: print-message
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    script: |
      #!/bin/sh
      echo "Message received from pipeline:"
      echo "${params.message}"
EOF

```

Simple CI/CD Pipeline

```

cat <<'EOF' | oc apply -f -
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: simple-cicd-pipeline
spec:

```

```

params:
- name: message
  type: string
tasks:
- name: build-stage
  taskRef:
    name: simple-message-task
  params:
    - name: message
      value: "Build stage completed"

- name: test-stage
  runAfter:
    - build-stage
  taskRef:
    name: simple-message-task
  params:
    - name: message
      value: "Test stage completed"

- name: deploy-stage
  runAfter:
    - test-stage
  taskRef:
    name: simple-message-task
  params:
    - name: message
      value: "Deploy stage completed"
EOF

```

PipelineRun creation - apply vs create issue

generateName tells OpenShift to generate a unique name. It cannot be used with oc apply because apply needs a fixed name to update the same object. Use oc create with generateName, or use metadata.name with oc apply.

```

# Correct with generateName
cat <<'EOF' | oc create -f -
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: simple-cicd-pipeline-run-
spec:
  pipelineRef:
    name: simple-cicd-pipeline
  params:
    - name: message
      value: "Starting CI/CD pipeline"
EOF

```

```

# Alternative fixed name with apply
metadata:
  name: simple-cicd-pipeline-run-1

```

Build-Test-Deploy using OpenShift CLI tasks

```

# Build task concept
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: openshift-build-task
spec:
  params:
    - name: buildconfig
      type: string
  steps:
    - name: start-build
      image: quay.io/openshift/origin-cli:latest
      script: |
        #!/bin/sh
        oc start-build $(params.buildconfig) --follow

# Test task can run mvn test, npm test, pytest or a sample shell test.
# Deploy task can run oc rollout restart/status deployment/<name>.

```

Git integration

```
Developer pushes code
-> GitHub/GitLab webhook
-> OpenShift EventListener
-> TriggerBinding extracts repo/branch/commit
-> TriggerTemplate creates PipelineRun
-> Pipeline executes build/test/deploy
```

13. DevOps and GitOps

Learning objective: Understand Jenkins integration, GitOps with Argo CD and image promotion across environments.

Concept flow

Traditional CI/CD: Git -> Jenkins/Pipeline -> Build -> Test -> Deploy
GitOps: Git desired state -> Argo CD watches -> Syncs OpenShift
Image promotion: Build once -> dev -> test -> prod

Jenkins integration concept

Jenkins can integrate with OpenShift using the oc CLI, OpenShift API, ServiceAccount token, kubeconfig, plugins and Git webhooks.

```
Git push -> Jenkins webhook -> Jenkins build/test -> Image push -> oc rollout or oc apply ->
OpenShift
```

```
oc create sa jenkins-deployer
oc policy add-role-to-user edit    system:serviceaccount:$(oc project -q):jenkins-deployer
```

```
oc auth can-i update deployment --as=system:serviceaccount:$(oc project
-q):jenkins-deployer
```

GitOps introduction with Argo CD

GitOps means Git is the source of truth for desired application and infrastructure state. Argo CD continuously compares Git desired state with the actual OpenShift cluster state and syncs differences.

```
Git Repository
deployment.yaml
service.yaml
route.yaml
-> Argo CD Application
-> Sync
-> OpenShift resources
```

GitOps Concept	Meaning
Application	Argo CD object representing one deployable app.
Sync	Apply Git state to cluster.
Drift	Difference between Git and cluster.
OutOfSync	Cluster state does not match Git.
Self-heal	Automatically correct manual cluster changes.
Prune	Delete cluster resources removed from Git.

Image promotion across environments

Build the image once, then promote the same tested image through dev, test and prod. This avoids rebuilding different artifacts for each environment.

```
oc tag pipeline-demo:latest pipeline-demo:dev
oc tag pipeline-demo:dev pipeline-demo:test
oc tag pipeline-demo:test pipeline-demo:prod
oc describe is pipeline-demo
```

```
# Across projects, if permitted:
```



```
oc tag myapp-dev/myapp:latest myapp-test/myapp:test
oc tag myapp-test/myapp:test myapp-prod/myapp:prod
```

14. Cloud-Native OpenShift

Learning objective: Understand OpenShift on public clouds, cluster upgrades, patching and operator-driven lifecycle.

Concept flow

Cloud Infrastructure -> OpenShift Cluster -> Operators -> Applications
Cluster updates -> CVO/MCO/Cluster Operators -> Nodes and platform components

OpenShift on AWS, Azure and GCP

Cloud	Compute	Storage	Load Balancer/DNS	Managed option examples
AWS	EC2	EBS/EFS	ELB/Route 53	ROSA, self-managed OCP
Azure	Azure VMs	Azure Disk/Azure Files	Azure Load Balancer/Azure DNS	ARO, self-managed OCP
GCP	Compute Engine	Persistent Disk/Filestore	Cloud Load Balancing/Cloud DNS	Self-managed or partner model

Cluster upgrades and patching basics

OpenShift upgrades are coordinated platform operations. They update cluster operators, control plane, nodes and platform services through a controlled update path.

```
# Admin-level commands, often restricted in Sandbox
oc get clusterversion
oc adm upgrade
oc adm upgrade --to=<version>
oc get clusteroperators
oc get nodes
```

Upgrade check	Why
Cluster operators healthy	Avoid upgrading a degraded platform.
Nodes ready	Ensure capacity during rolling node updates.
PDBs valid	Avoid blocking node drains.
Deprecated APIs checked	Avoid workloads failing after upgrade.
Operators compatible	Third-party operators must support target version.
Backups planned	Critical for production/self-managed clusters.

Operator-driven lifecycle

Operators are Kubernetes-native controllers that know how to install, configure, upgrade, repair and operate complex applications or platform components.

```
OperatorHub -> Install Operator -> Subscription -> InstallPlan -> CSV
-> CRD available
-> User creates Custom Resource
-> Operator reconciles desired state
```

Operator object	Meaning
OperatorHub	Catalog UI for discovering Operators.
Subscription	Tracks selected package/channel.

Operator object	Meaning
InstallPlan	Plan for installing or updating Operator resources.
CSV	ClusterServiceVersion metadata for Operator version.
CRD	New API type added by Operator.
Custom Resource	User-created instance managed by Operator.

15. Optimization and Cost Awareness

Learning objective: Right-size resources, tune performance and control OpenShift cost and quota usage.

Concept flow

Measure -> Analyze -> Right-size -> Automate -> Monitor continuously

Resource optimization

Optimization is not about using the smallest possible resource. It is about using the right amount of CPU, memory, storage and replicas for the workload.

Optimization area	Action
CPU	Set realistic requests and limits. Watch throttling.
Memory	Avoid too-low limits that cause OOMKilled.
Replicas	Use enough for availability but avoid idle over-scaling.
Images	Use minimal trusted images and avoid unnecessary tools.
Storage	Request right PVC size and delete unused PVCs carefully.
Logs	Avoid excessive debug logs and sensitive values.

Performance tuning basics

Application slow?

1. `oc get pods`
2. `oc describe pod <pod>`
3. `oc logs <pod>`
4. `oc top pods`
5. Check CPU throttling, memory limits, restarts, route/service endpoints
6. Tune resource request/limit, replica count, probes, app config
7. Monitor again

Cost control strategies

Strategy	Why it helps
ResourceQuota	Prevents unlimited project growth.
LimitRange	Applies default requests/limits if developers forget.
HPA max replicas	Prevents autoscaling cost explosion.
Cleanup policy	Removes unused deployments, services, routes, PVCs and old ReplicaSets.
Labels for owner/cost-center	Improves cost allocation and ownership.
Log retention	Controls logging storage and processing cost.

```
# Useful quota/resource cleanup checks
oc get all
oc get svc --no-headers | wc -l
```

```
oc get route --no-headers | wc -l
oc get rs --no-headers | wc -l
oc describe appliedclusterresourcequota
```

```
# Delete old zero ReplicaSets
oc get rs --no-headers | awk '$2==0 && $3==0 && $4==0 {print $1}' | xargs -r oc delete rs
```

16. Troubleshooting Appendix - Errors Discussed in the Lab

Learning objective: Quickly recognize common errors from the hands-on work and apply the correct fix.

Concept flow

Error message -> Meaning -> Check describe/events/quota -> Fix -> Retry

Error/Symptom	Meaning	Fix
PROJECT: command not found	Spaces around = in Bash variable assignment.	Use PROJECT=\$(oc project -q).
CreateContainerConfigError for config pod	ConfigMap or Secret referenced by pod is missing or invalid.	oc describe pod; create missing ConfigMap/Secret.
buildconfig already exists	You reran oc new-build with an existing name.	Run oc start-build or delete old bc/is first.
timed out waiting for build to run	Build object created but did not start quickly.	oc get builds; oc describe build; oc logs build/name.
GitHub webhook missing scheme	Payload URL does not start with http:// or https://.	Copy full OpenShift webhook URL.
Route kind service invalid	kind is case-sensitive.	Use kind: Service.
Quota exceeded services 30/30	Project service quota full.	Delete unused services, then retry expose/new-app.
Quota replicasets 30/30	Too many Deployment ReplicaSets.	Delete old deployments or zero ReplicaSets.
Web Terminal stuck	Workspace pod or quota/browser issue.	Delete workspace pod, refresh, or use local oc login.
oc not found inside container	You are inside app container.	Run Linux commands or exit back to OpenShift terminal.
cannot use generate name with apply	generateName requires create, not apply.	Use oc create -f - or fixed metadata.name.

Quota cleanup commands

```
# List resources
oc get deployments
oc get pods
oc get svc
oc get route
oc get rs
oc get rc
oc get pvc

# Safe examples - adjust names based on active labs
oc delete deployment config-demo-env config-demo-volume secret-demo-env secret-demo-volume
--ignore-not-found
oc delete deployment vk-blue-app vk-green-app blue-app green-app public-demo docker-demo-app
--ignore-not-found
oc delete svc network-demo-svc network-demo-nodeport vk-blue-svc vk-green-svc blue-svc
green-svc --ignore-not-found
oc delete route network-demo-route vk-bg-route bg-route docker-demo-app hello-s2i
--ignore-not-found

# Delete unused old rollout histories with 0 desired/current/ready
```

```
oc get rs --no-headers | awk '$2==0 && $3==0 && $4==0 {print $1}' | xargs -r oc delete rs
oc get rc --no-headers | awk '$2==0 && $3==0 && $4==0 {print $1}' | xargs -r oc delete rc
```

Final end-to-end OpenShift mental model

Developer writes code

- > Git repository
- > BuildConfig/OpenShift Pipeline/Jenkins builds image
- > Image pushed to internal/external registry
- > ImageStream tracks image tags
- > Deployment/DeploymentConfig rolls out pods
- > Service gives stable internal access
- > Route exposes app externally
- > ConfigMaps/Secrets provide runtime config
- > PVC provides persistent storage when needed
- > RBAC/SCC controls permissions and security
- > Monitoring/logging/alerts support operations
- > GitOps and image promotion control releases
- > Optimization and quota management control cost

End of Learner Guide

This guide consolidates the OpenShift concepts, labs, troubleshooting and best practices discussed throughout the training conversation.

Recommended next practice:

1. Clean current Sandbox quota.
2. Re-run each module lab in sequence.
3. Maintain one command history file per day.
4. Build one mini-project combining build, config, service, route, PVC, monitoring and pipeline.
5. Convert the mini-project into a trainer demo.

Prepared by: Vivek Arora

Senior Technical Trainer & Author/Consultant